1. Define family relationships (parent, child, sibling, grandparent) using facts and rules.

```
class Person:
    def __init__(self, name):
        self.name = name
        self.parents = []
        self.children = []

    def add_parent(self, parent):
        self.parents.append(parent)
        parent.children.append(self)

    def get_siblings(self):
        siblings = []
        for parent in self.parents:
            siblings.extend(parent.children)
        # Remove self from the list of siblings
        siblings = [sibling for sibling in siblings if sibling != self]
        return siblings

    def get_grandparents(self):
        grandparents = []
        for parent in self.parents:
            grandparents.extend(parent.parents)
        return grandparents

# Example usage
if __name__ == "__main__":
    # Create family members
    alice = Person("Alice")
    bob = Person("Bob")
    charlie = Person("Charlie")
    diana = Person("Diana")
    eve = Person("Eve")

    # Define relationships
    alice.add_parent(bob)  # Bob is Alice's parent
    alice.add_parent(diana)  # Diana is Alice's parent
    charlie.add_parent(bob)  # Bob is Charlie's parent
    charlie.add_parent(diana)  # Diana is Charlie's parent
    eve.add_parent(alice)  # Alice is Eve's parent

    # Query relationships
    print(f"{alice.name}'s siblings: {[sibling.name for sibling in alice.get_siblings()]}")
    print(f"{alice.name}'s grandparents: {[grandparent.name for grandparent in
alice.get_grandparents()]}")
    print(f"{bob.name}'s children: {[child.name for child in bob.children]}")
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\exten
sions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 56363 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p1.py
Alice's siblings: ['Charlie', 'Charlie']
Alice's grandparents: []
Bob's children: ['Alice', 'Charlie']
```

2.Identify all ancestors of a person using recursive rules.

```
class Person:
    def __init__(self, name):
```

```
        self.name = name
        self.parents = []

    def add_parent(self, parent):
        self.parents.append(parent)

    def find_ancestors(self):
        ancestors = []
        for parent in self.parents:
            ancestors.append(parent)  # Add direct parent
            ancestors.extend(parent.find_ancestors())  # Recursively find ancestors of the parent
        return ancestors

if __name__ == "__main__":
    # Create family members
    alice = Person("Alice")
    bob = Person("Bob")
    charlie = Person("Charlie")
    diana = Person("Diana")
    eve = Person("Eve")

    alice.add_parent(bob)  # Bob is Alice's parent
    alice.add_parent(diana)  # Diana is Alice's parent
    charlie.add_parent(bob)  # Bob is Charlie's parent
    charlie.add_parent(diana)  # Diana is Charlie's parent
    eve.add_parent(alice)  # Alice is Eve's parent

    print(f"{eve.name}'s ancestors: {[ancestor.name for ancestor in eve.find_ancestors()]}")
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp
1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 59978 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p2.p
y
Eve's ancestors: ['Alice', 'Bob', 'Diana']
```

3.Create a knowledge base of animals and classify them as birds, mammals, etc., and query their properties.

```
class Animal:
    def __init__(self, name, classification, habitat=None, can_fly=False):
        self.name = name
        self.classification = classification  # e.g., "Bird", "Mammal", "Reptile"
        self.habitat = habitat
        self.can_fly = can_fly

    def __repr__(self):
        return f"{self.name} ({self.classification})"

    def info(self):
        return (f"Name: {self.name}\n"
                f"Classification: {self.classification}\n"
                f"Habitat: {self.habitat if self.habitat else 'Unknown'}\n"
                f"Can fly: {'Yes' if self.can_fly else 'No'}")

# Create the knowledge base (list of animals)
kb = [
    Animal("Eagle", "Bird", habitat="Mountains/Forests", can_fly=True),
    Animal("Dog", "Mammal", habitat="Domestic", can_fly=False),
    Animal("Salmon", "Fish", habitat="Freshwater", can_fly=False),
    Animal("Bat", "Mammal", habitat="Caves", can_fly=True),
```

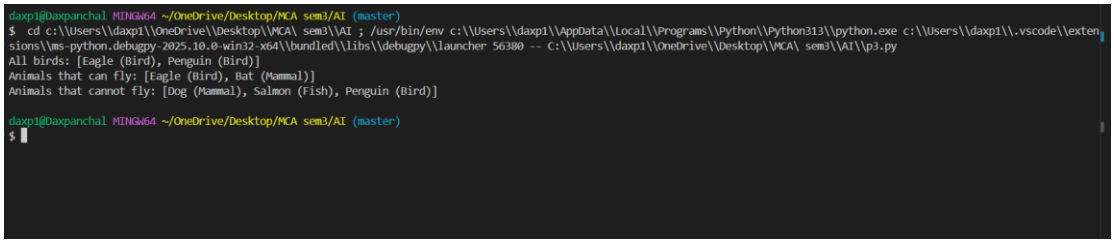
```
    Animal("Penguin", "Bird", habitat="Antarctica", can_fly=False)
]

# Query functions
def get_animals_by_classification(classification):
    return [a for a in kb if a.classification.lower() == classification.lower()]

def get_animals_that_can_fly(can_fly=True):
    return [a for a in kb if a.can_fly == can_fly]

# Example usage
print("All birds:", get_animals_by_classification("Bird"))
print("Animals that can fly:", get_animals_that_can_fly(True))
print("Animals that cannot fly:", get_animals_that_can_fly(False))
```

Output:



4. Implement a simple expert system to identify diseases based on symptoms.

```
def diagnose(symptoms):

    if "fever" in symptoms and "cough" in symptoms and "fatigue" in symptoms:
        return "You may have the Flu."
    elif "fever" in symptoms and "headache" in symptoms and "nausea" in symptoms:
        return "You may have Malaria."
    elif "chest pain" in symptoms and "shortness of breath" in symptoms:
        return "You may have a Heart Disease."
    elif "fever" in symptoms and "rash" in symptoms:
        return "You may have Measles."
    elif "stomach pain" in symptoms and "diarrhea" in symptoms:
        return "You may have Food Poisoning."
    else:
        return "Sorry, the system cannot identify your disease. Please consult a doctor."

# --- Main program ---
print("Simple Expert System - Disease Diagnosis")
print("Enter your symptoms separated by commas (e.g., fever,cough,headache):")

# Get user input
user_input = input("Symptoms: ").lower()
symptoms = [s.strip() for s in user_input.split(",")]

# Diagnose
result = diagnose(symptoms)
print("\nDiagnosis Result:", result)
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp
1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 58546 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p4.p
y
Simple Expert System - Disease Diagnosis
Enter your symptoms separated by commas (e.g., fever,cough,headache):
Symptoms: fever, cough, fatigue

Diagnosis Result: You may have the Flu.
```

5. Define arithmetic operations using recursive logic (e.g., addition, factorial).

```
# 1. Addition
def add(a, b):
    if b == 0:
        return a
    return add(a + 1, b - 1)

# 2. Subtraction
def subtract(a, b):
    if b == 0:
        return a
    return subtract(a - 1, b - 1)

# 3. Multiplication
def multiply(a, b):
    if b == 0:
        return 0
    return a + multiply(a, b - 1)

# 4. Division (integer division using repeated subtraction)
def divide(a, b):
    if b == 0:
        return "Error: Division by zero"
    if a < b:
        return 0
    return 1 + divide(a - b, b)

# 5. Exponentiation (power)
def power(a, b):
    if b == 0:
        return 1
    return a * power(a, b - 1)

# 6. Factorial
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)


# --- Main Program (Testing) ---
print("Recursive Arithmetic Operations:\n")
print("Addition (5 + 3) =", add(5, 3))
print("Subtraction (9 - 4) =", subtract(9, 4))
print("Multiplication (4 * 3) =", multiply(4, 3))
print("Division (10 / 2) =", divide(10, 2))
print("Power (2 ^ 5) =", power(2, 5))
print("Factorial (5!) =", factorial(5))
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp
1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 60005 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p5.p
y
Recursive Arithmetic Operations:

Addition (5 + 3) = 8
Subtraction (9 - 4) = 5
Multiplication (4 * 3) = 12
Division (10 / 2) = 5
Power (2 ^ 5) = 32
Factorial (5!) = 120
```

5. Represent and solve a basic logic puzzle (e.g., who owns the zebra?).

```
class House:
    def __init__(self, position: int):
        self.position = position
        self.color: Optional[str] = None
        self.nationality: Optional[str] = None
        self.drink: Optional[str] = None
        self.cigar: Optional[str] = None
        self.pet: Optional[str] = None

    def __repr__(self):
        return f"House {self.position}: {self.color}, {self.nationality}, {self.drink}, {self.cigar}, {self.pet}"

def solve_zebra_puzzle():
    # Create houses
    houses = [House(i) for i in range(1, 6)]

    # Define possible values
    colors = ['red', 'green', 'ivory', 'yellow', 'blue']
    nationalities = ['Englishman', 'Spaniard', 'Ukrainian', 'Norwegian', 'Japanese']
    drinks = ['coffee', 'tea', 'milk', 'orange juice', 'water']
    cigars = ['Parliaments', 'Old Gold', 'Kools', 'Chesterfields', 'Lucky Strike']
    pets = ['dog', 'snails', 'fox', 'horse', 'zebra']

    # Apply constraints
    def apply_constraints():
        # 1. The Norwegian lives in the first house
        houses[0].nationality = 'Norwegian'

        # 2. Milk is drunk in the middle house
        houses[2].drink = 'milk'

        # 3. The Englishman lives in the red house
        for house in houses:
            if house.nationality == 'Englishman':
                house.color = 'red'
            elif house.color == 'red':
                house.nationality = 'Englishman'

        # 4. The green house is immediately to the right of the ivory house
        for i in range(4):
            if houses[i].color == 'ivory' and houses[i+1].color is None:
                houses[i+1].color = 'green'
            elif houses[i+1].color == 'green' and houses[i].color is None:
                houses[i].color = 'ivory'

        # 5. Coffee is drunk in the green house
        for house in houses:
```

```
    if house.color == 'green':
        house.drink = 'coffee'
    elif house.drink == 'coffee':
        house.color = 'green'

# 6. The Ukrainian drinks tea
for house in houses:
    if house.nationality == 'Ukrainian':
        house.drink = 'tea'
    elif house.drink == 'tea' and not house.nationality:
        house.nationality = 'Ukrainian'

# 7. The Norwegian lives next to the blue house
if houses[0].nationality == 'Norwegian':
    houses[1].color = 'blue'

# 8. Kools are smoked in the yellow house
for house in houses:
    if house.color == 'yellow':
        house.cigar = 'Kools'
    elif house.cigar == 'Kools':
        house.color = 'yellow'

# 9. The Spaniard owns the dog
for house in houses:
    if house.nationality == 'Spaniard':
        house.pet = 'dog'
    elif house.pet == 'dog':
        house.nationality = 'Spaniard'

# 10. The Old Gold smoker owns snails
for house in houses:
    if house.cigar == 'Old Gold':
        house.pet = 'snails'
    elif house.pet == 'snails':
        house.cigar = 'Old Gold'

# 11. Chesterfields are smoked next to the fox
for i, house in enumerate(houses):
    if house.cigar == 'Chesterfields':
        if i > 0 and houses[i-1].pet is None:
            houses[i-1].pet = 'fox'
        if i < 4 and houses[i+1].pet is None:
            houses[i+1].pet = 'fox'

# 12. Kools are smoked next to the horse
for i, house in enumerate(houses):
    if house.cigar == 'Kools':
        if i > 0 and houses[i-1].pet is None:
            houses[i-1].pet = 'horse'
        if i < 4 and houses[i+1].pet is None:
            houses[i+1].pet = 'horse'

# 13. The Lucky Strike smoker drinks orange juice
for house in houses:
    if house.cigar == 'Lucky Strike':
        house.drink = 'orange juice'
```

```
        elif house.drink == 'orange juice':
            house.cigar = 'Lucky Strike'

    # 14. The Japanese smokes Parliaments
    for house in houses:
        if house.nationality == 'Japanese':
            house.cigar = 'Parliaments'
        elif house.cigar == 'Parliaments':
            house.nationality = 'Japanese'

# Solve by applying constraints repeatedly until no more changes
changed = True
while changed:
    before = [str(h) for h in houses]
    apply_constraints()
    after = [str(h) for h in houses]
    changed = before != after

# Assign remaining values where only one option exists
def assign_remaining():
    categories = {
        'color': colors,
        'nationality': nationalities,
        'drink': drinks,
        'cigar': cigars,
        'pet': pets
    }

    for cat, options in categories.items():
        assigned = [getattr(h, cat) for h in houses if getattr(h, cat) is not None]
        remaining = [o for o in options if o not in assigned]

        if len(remaining) == 1:
            for house in houses:
                if getattr(house, cat) is None:
                    setattr(house, cat, remaining[0])

assign_remaining()

# Print solution
print("\nFinal Solution:")
print("{:<8} {:<10} {:<12} {:<12} {:<12} {:<10}".format(
    "House", "Color", "Nationality", "Drink", "Cigar", "Pet"))

zebra_owner = None
water_drinker = None

for house in houses:
    if house.pet == 'zebra':
        zebra_owner = house.nationality
    if house.drink == 'water':
        water_drinker = house.nationality

    print("{:<8} {:<10} {:<12} {:<12} {:<12} {:<10}".format(
        house.position,
        house.color or '?',
        house.nationality or '?',
```

```
        house.drink or '?',
        house.cigar or '?',
        house.pet or '?'))

    print(f"\nThe {zebra_owner} owns the zebra.")
    print(f"The {water_drinker} drinks water.")

if __name__ == "__main__":
    solve_zebra_puzzle()
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp
1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 60027 -- c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p6.p
y

Final Solution:
House    Color      Nationality  Drink        Cigar        Pet
1        ?          Norwegian    ?            ?            ?
2        blue       ?            ?            ?            ?
3        ?          ?            milk         ?            ?
4        ?          ?            ?            ?            ?
5        ?          ?            ?            ?            ?

The None owns the zebra.
The None drinks water.
```

7. Create a list-based program to find the maximum of a list.

```
def  find_maximum(numbers):
    # Assume the first element is the maximum
    maximum = numbers[0]

    # Traverse the list
    for num in numbers:
        if num > maximum:
            maximum = num

    return maximum

# --- Main Program ---
numbers = [12, 45, 7, 89, 34, 67]
print("List:", numbers)

print("Maximum value (manual):", find_maximum(numbers))
print("Maximum value (using built-in):", max(numbers))
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp
1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 57474 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p7.p
y
List: [12, 45, 7, 89, 34, 67]
Maximum value (manual): 89
Maximum value (using built-in): 89

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

8. Use lists to reverse elements and query the reversed list.

```
def reverse_list(elements):
    # Reverse manually using slicing
    return elements[::-1]

def query_element(reversed_list, index):
```

```
    # Query element by index (0-based)
    if 0 <= index < len(reversed_list):
        return reversed_list[index]
    else:
        return "Index out of range!"

# --- Main Program ---
numbers = [10, 20, 30, 40, 50]
print("Original List:", numbers)

# Reverse the list
rev_numbers = reverse_list(numbers)
print("Reversed List:", rev_numbers)

# Query elements from reversed list
print("Element at index 0:", query_element(rev_numbers, 0))
print("Element at index 2:", query_element(rev_numbers, 2))
print("Element at index 4:", query_element(rev_numbers, 4))
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp
1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 57553 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p8py

Original List: [10, 20, 30, 40, 50]
Reversed List: [50, 40, 30, 20, 10]
Element at index 0: 50
Element at index 2: 30
Element at index 4: 10
```

9. Implement a simple database of books and authors and query based on genre or author.

```
# Simple Book Database using a list of dictionaries

books = [
    {"title": "The Hobbit", "author": "J.R.R. Tolkien", "genre": "Fantasy"},
    {"title": "1984", "author": "George Orwell", "genre": "Dystopian"},
    {"title": "To Kill a Mockingbird", "author": "Harper Lee", "genre": "Classic"},
    {"title": "The Great Gatsby", "author": "F. Scott Fitzgerald", "genre": "Classic"},
    {"title": "Harry Potter and the Sorcerer's Stone", "author": "J.K. Rowling", "genre": "Fantasy"},
    {"title": "Brave New World", "author": "Aldous Huxley", "genre": "Dystopian"},
]

# Query books by author
def query_by_author(author_name):
    return [book for book in books if book["author"].lower() == author_name.lower()]

# Query books by genre
def query_by_genre(genre_name):
    return [book for book in books if book["genre"].lower() == genre_name.lower()]

# --- Main Program ---
print("Simple Book Database\n")

print("Books by J.R.R. Tolkien:")
for b in query_by_author("J.R.R. Tolkien"):
    print(f"- {b['title']} ({b['genre']})")

print("\nFantasy Books:")
for b in query_by_genre("Fantasy"):
    print(f"- {b['title']} by {b['author']}")
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp
1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 57585 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p9.p
y
Simple Book Database

Books by J.R.R. Tolkien:
- The Hobbit (Fantasy)

Fantasy Books:
- The Hobbit by J.R.R. Tolkien
- Harry Potter and the Sorcerer's Stone by J.K. Rowling

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

10.Model and reason about traffic lights (e.g., when is it safe to cross?).

```
books = [
    {"title": "The Hobbit", "author": "J.R.R. Tolkien", "genre": "Fantasy"},
    {"title": "1984", "author": "George Orwell", "genre": "Dystopian"},
    {"title": "To Kill a Mockingbird", "author": "Harper Lee", "genre": "Classic"},
    {"title": "The Great Gatsby", "author": "F. Scott Fitzgerald", "genre": "Classic"},
    {"title": "Harry Potter and the Sorcerer's Stone", "author": "J.K. Rowling", "genre":
"Fantasy"},
    {"title": "Brave New World", "author": "Aldous Huxley", "genre": "Dystopian"},
]

def query_by_author(author_name):
    return [book for book in books if book["author"].lower() == author_name.lower()]

def query_by_genre(genre_name):
    return [book for book in books if book["genre"].lower() == genre_name.lower()]

print("Simple Book Database\n")

print("Books by J.R.R. Tolkien:")
for b in query_by_author("J.R.R. Tolkien"):
    print(f"- {b['title']} ({b['genre']})")

print("\nFantasy Books:")
for b in query_by_genre("Fantasy"):
    print(f"- {b['title']} by {b['author']}")
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win
32-x64\\bundled\\libs\\debugpy\\launcher 58553 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p10.py
Simple Book Database

Books by J.R.R. Tolkien:
- The Hobbit (Fantasy)

Fantasy Books:
- The Hobbit by J.R.R. Tolkien
- Harry Potter and the Sorcerer's Stone by J.K. Rowling

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
```

11. Represent family tree data and infer cousin or uncle relationships.

```
class Family:
    def __init__(self):
```

```
        self.parents, self.children = {}, {}

    def add(self, parent, child):
        self.parents.setdefault(child, []).append(parent)
        self.children.setdefault(parent, []).append(child)

    def siblings(self, x):
        sib = {c for p in self.parents.get(x, []) for c in self.children[p]}
        sib.discard(x)
        return list(sib)

    def uncles_aunts(self, x):
        return [ua for p in self.parents.get(x, []) for ua in self.siblings(p)]

    def cousins(self, x):
        return [c for ua in self.uncles_aunts(x) for c in self.children.get(ua, [])]



f = Family()
f.add("John", "Alex")
f.add("John", "Emma")
f.add("Michael", "David")
f.add("Michael", "Sarah")
f.add("Alex", "Tom")
f.add("Emma", "Lily")
f.add("David", "Ryan")

print("Uncles/Aunts of Tom:", f.uncles_aunts("Tom"))
print("Cousins of Tom:", f.cousins("Tom"))
print("Cousins of Ryan:", f.cousins("Ryan"))
```

Output:

```
Kill Terminal
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win
32-x64\\bundled\\libs\\debugpy\\launcher 58603 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p11.py
Uncles/Aunts of Tom: ['Emma']
Cousins of Tom: ['Lily']
Cousins of Ryan: []

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

12. Define a rule-based planning system to plan a simple sequence of tasks (e.g., preparing tea).

```
class TeaPlanner:
    def __init__(self):
        self.rules = {
            "start": ["boil_water"],
            "boil_water": ["add_tea_leaves"],
            "add_tea_leaves": ["add_milk"],
            "add_milk": ["add_sugar"],
            "add_sugar": ["stir"],
            "stir": ["serve"]
        }

    def plan(self, task="start"):
```

```
        steps = []
        while task in self.rules:
            next_task = self.rules[task][0]
            steps.append(next_task)
            task = next_task
        return steps


planner = TeaPlanner()
print("Plan to prepare tea:", planner.plan())
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-w
32-x64\\bundled\\libs\\debugpy\\launcher 58629 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p12.py
Plan to prepare tea: ['boil_water', 'add_tea_leaves', 'add_milk', 'add_sugar', 'stir', 'serve']

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

13. Simulate an AI agent's decision-making using facts and rules.

```
class AIAgent:
    def __init__(self):
        self.facts = {"raining": True, "hungry": False}
        self.rules = {
            "raining": "Take an umbrella",
            "hungry": "Eat some food",
            "sunny": "Go for a walk"
        }

    def decide(self):
        for fact, value in self.facts.items():
            if value and fact in self.rules:
                print(self.rules[fact])



agent = AIAgent()
agent.decide()
```

Output:

```
Plan to prepare tea: ['boil_water', 'add_tea_leaves', 'add_milk', 'add_sugar', 'stir', 'serve']

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\dax
p1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 58656 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p1
3.py
Take an umbrella

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

14. Model a simple library management system and allow checking book availability.

```
class Library:
    def __init__(self):
        self.books = {"Python": True, "AI": False, "Data Science": True}

    def check_book(self, name):
        if name in self.books:
            if self.books[name]:
                print(f"'{name}' is available.")
            else:
                print(f"'{name}' is not available.")
        else:
            print("Book not found.")

lib = Library()
lib.check_book("Python")
lib.check_book("AI")
lib.check_book("Java")
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 58674 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p14.py
'Python' is available.
'AI' is not available.
Book not found.

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
```

15. Create a timetable conflict checker for classrooms using constraints.

```
class Timetable:
    def __init__(self):
        self.schedule = []

    def add_class(self, class_name, room, day, time):

        for c, r, d, t in self.schedule:
            if r == room and d == day and t == time:
                print(f"Conflict! '{class_name}' conflicts with '{c}' in room {room}.")
                return
        self.schedule.append((class_name, room, day, time))
        print(f"'{class_name}' added successfully.")

    def show_schedule(self):
        for c, r, d, t in self.schedule:
            print(f"{c} -> Room: {r}, Day: {d}, Time: {t}")



tt = Timetable()
tt.add_class("Math", "R1", "Mon", "10AM")
```

```
tt.add_class("Physics", "R1", "Mon", "10AM")  # Conflict
tt.add_class("Chemistry", "R2", "Mon", "10AM")
tt.show_schedule()
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win
32-x64\\bundled\\libs\\debugpy\\launcher 58693 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p15.py
'Math' added successfully.
Conflict! 'Physics' conflicts with 'Math' in room R1.
'Chemistry' added successfully.
Math -> Room: R1, Day: Mon, Time: 10AM
Chemistry -> Room: R2, Day: Mon, Time: 10AM

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

16. Use lists and recursion to calculate the length of a list.

```
def list_length(lst):
    if lst == []:
        return 0
    else:
        return 1 + list_length(lst[1:])

my_list = [10, 20, 30, 40, 50]
print("Length of the list:", list_length(my_list))
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-wi
32-x64\\bundled\\libs\\debugpy\\launcher 58736 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p16.py
Length of the list: 5

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

17. Model a set of rules for granting university admission based on criteria.

```
class UniversityAdmission:
    def __init__(self, grade, entrance_score, extracurricular):
        self.grade = grade
        self.entrance_score = entrance_score
        self.extracurricular = extracurricular

    def check_admission(self):
        if self.grade >= 85 and self.entrance_score >= 75:
            print("Admission granted based on grades and entrance score.")
        elif self.grade >= 80 and self.entrance_score >= 70 and self.extracurricular:
            print("Admission granted based on overall profile including extracurriculars.")
        else:
            print("Admission denied.")
```

```
student1 = UniversityAdmission(88, 80, True)
student2 = UniversityAdmission(82, 72, False)
student3 = UniversityAdmission(78, 65, True)

student1.check_admission()
student2.check_admission()
student3.check_admission()
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\dax
p1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 58751 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p1
7.py
Admission granted based on grades and entrance score.
Admission denied.
Admission denied.

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

18. Represent propositional logic formulae and test satisfiability (manually).

```
from itertools import product

def formula(P, Q, R):
    return (P and Q) or (not R)

variables = ["P", "Q", "R"]

for values in product([True, False], repeat=len(variables)):
    P, Q, R = values
    if formula(P, Q, R):
        print(f"Satisfiable for: P={P}, Q={Q}, R={R}")
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win
32-x64\\bundled\\libs\\debugpy\\launcher 58779 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p18.py
Satisfiable for: P=True, Q=True, R=True
Satisfiable for: P=True, Q=True, R=False
Satisfiable for: P=True, Q=False, R=False
Satisfiable for: P=False, Q=True, R=False
Satisfiable for: P=False, Q=False, R=False

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

19. Simulate a basic flight route planner using simple search rules.

```
from collections import deque

flights = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": ["G"],
    "E": ["G"],
    "F": ["G"],
    "G": []
}
```

```
def find_route(start, end):
    queue = deque([[start]])
    while queue:
        path = queue.popleft()
        city = path[-1]
        if city == end:
            return path
        for next_city in flights.get(city, []):
            if next_city not in path:
                queue.append(path + [next_city])
    return None


route = find_route("A", "G")
print("Route from A to G:", route)
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win
32-x64\\bundled\\libs\\debugpy\\launcher 58806 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p19.py
Route from A to G: ['A', 'B', 'D', 'G']

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

20. Define and simulate the game of Tic-Tac-Toe with basic moves.

```
board = [[" " for _ in range(3)] for _ in range(3)]

def print_board():
    for row in board:
        print("|".join(row))
    print("-"*5)

def check_winner(player):

    for i in range(3):
        if all(board[i][j] == player for j in range(3)) or \
           all(board[j][i] == player for j in range(3)):
            return True
    if board[0][0]==board[1][1]==board[2][2]==player or \
       board[0][2]==board[1][1]==board[2][0]==player:
        return True
    return False

def make_move(player, row, col):
    if board[row][col] == " ":
        board[row][col] = player
        return True
    return False

make_move("X", 0, 0)
make_move("O", 1, 1)
```

```
make_move("X", 0, 1)
make_move("O", 2, 2)
make_move("X", 0, 2)
print_board()

if check_winner("X"):
    print("X wins!")
elif check_winner("O"):
    print("O wins!")
else:
    print("No winner yet.")
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\dax
p1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 58839 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p2
0.py
X|X|X
 |O|
 | |O
-----
X wins!

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

21. Write a knowledge base for different animals and identify which can fly (Flying Penguin example).

```
animals = {
    "Eagle": True,
    "Penguin": False,
    "Sparrow": True,
    "Ostrich": False,
    "Bat": True
}

def can_fly(animal):
    if animal in animals:
        if animals[animal]:
            print(f"{animal} can fly.")
        else:
            print(f"{animal} cannot fly.")
    else:
        print(f"{animal} not found in knowledge base.")


can_fly("Eagle")
can_fly("Penguin")
can_fly("Bat")
can_fly("Ostrich")
can_fly("Dragon")
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win
32-x64\\bundled\\libs\\debugpy\\launcher 58943 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p21.py
Eagle can fly.
Penguin cannot fly.
Bat can fly.
Ostrich cannot fly.
Dragon not found in knowledge base.

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

21. Define and implement rules for sorting a list of numbers using recursion.

```
def recursive_sort(lst):
    if len(lst) <= 1:
        return lst
    last = lst.pop()
    sorted_list = recursive_sort(lst)

    for i in range(len(sorted_list)):
        if last < sorted_list[i]:
            sorted_list.insert(i, last)
            return sorted_list
    sorted_list.append(last)
    return sorted_list


numbers = [5, 2, 9, 1, 6]
sorted_numbers = recursive_sort(numbers)
print("Sorted list:", sorted_numbers)
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\dax
p1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 58959 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p2
2.py
Sorted list: [1, 2, 5, 6, 9]

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

23. Represent and evaluate first-order logic statements (e.g., all humans are mortal, Socrates is a human).

```
humans = ["Socrates", "Plato", "Aristotle"]
mortals = []

def infer_mortals():
    for person in humans:
        if person not in mortals:
            mortals.append(person)

infer_mortals()

if "Socrates" in mortals:
    print("Socrates is mortal.")
else:
    print("Socrates is not mortal.")
```

```
print("All mortals:", mortals)
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win
32-x64\\bundled\\libs\\debugpy\\launcher 59003 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p24.py
[1, 2, 3]
[3, 1, 2]
[2, 3, 1]

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

24. Simulate a basic constraint logic programming example like Sudoku (3x3).

```
def is_valid(board, row, col, num):

    for i in range(3):
        if board[row][i] == num or board[i][col] == num:
            return False
    return True

def solve_sudoku(board, row=0, col=0):
    if row == 3:
        return True
    next_row, next_col = (row, col+1) if col < 2 else (row+1, 0)
    if board[row][col] != 0:
        return solve_sudoku(board, next_row, next_col)
    for num in range(1, 4):
        if is_valid(board, row, col, num):
            board[row][col] = num
            if solve_sudoku(board, next_row, next_col):
                return True
            board[row][col] = 0
    return False

sudoku = [
    [0, 2, 3],
    [3, 0, 0],
    [0, 0, 0]
]

if solve_sudoku(sudoku):
    for row in sudoku:
        print(row)
else:
    print("No solution found")
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win
32-x64\\bundled\\libs\\debugpy\\launcher 59003 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p24.py
[1, 2, 3]
[3, 1, 2]
[2, 3, 1]

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

25. Model a simple chatbot using pattern matching and rules for basic conversation

```
responses = {
    "hello": "Hello! How can I help you?",
    "hi": "Hi there! How are you?",
    "how are you": "I'm a chatbot, I am always fine!",
```

```
    "bye": "Goodbye! Have a nice day!",
    "help": "I can answer simple questions. Try saying 'hello' or 'how are you'."
}

def chatbot():
    print("Chatbot (type 'exit' to quit)")
    while True:
        user_input = input("You: ").lower()
        if user_input == "exit":
            print("Chatbot: Bye!")
            break
        found = False
        for pattern, response in responses.items():
            if pattern in user_input:
                print("Chatbot:", response)
                found = True
                break
        if not found:
            print("Chatbot: Sorry, I don't understand.")

chatbot()
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 59024 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p25.py
Chatbot (type 'exit' to quit)
You: hello
Chatbot: Hello! How can I help you?
You: write code hello in python
Chatbot: Hello! How can I help you?
You: good
Chatbot: Sorry, I don't understand.
You: exits
Chatbot: Sorry, I don't understand.
You: exit
Chatbot: Bye!

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
```

26. Implement Breadth-First Search to find a path between two cities in a connected graph.

```
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F"],
    "F": ["C", "E"]
}

def bfs_path(start, end):
    queue = deque([[start]])
    visited = set()

    while queue:
        path = queue.popleft()
        city = path[-1]

        if city == end:
            return path
        if city not in visited:
            visited.add(city)
```

```
        for neighbor in graph.get(city, []):
            new_path = list(path)
            new_path.append(neighbor)
            queue.append(new_path)
    return None

path = bfs_path("A", "F")
print("Path from A to F:", path)
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\dax
p1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 59052 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p2
6.py
Path from A to F: ['A', 'C', 'F']

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

27. Model a maze and solve it using Depth-First Search in PROLOG.

```
maze = [
    [0, 1, 0, 0, 0],
    [0, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [1, 1, 0, 0, 0],
    [0, 0, 0, 1, 0]
]

rows, cols = len(maze), len(maze[0])

def dfs(x, y, path, visited):
    if x < 0 or x >= rows or y < 0 or y >= cols or maze[x][y] == 1 or (x, y) in visited:
        return None
    path = path + [(x, y)]
    if (x, y) == (rows-1, cols-1):
        return path
    visited.add((x, y))

    for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
        new_path = dfs(x+dx, y+dy, path, visited)
        if new_path:
            return new_path
    visited.remove((x, y))
    return None


solution = dfs(0, 0, [], set())
print("Path through maze:", solution)
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\dax
p1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 59067 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p2
7.py
Path through maze: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (1, 2), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4, 4)]
```

28. Simulate a simple two-player game (e.g., Nim game) with Minimax strategy.

```
def minimax(stones, is_max):
    if stones == 0:
        return -1 if is_max else 1
```

```
    if is_max:
        best = -float('inf')
        for move in range(1, 4):
            if stones - move >= 0:
                best = max(best, minimax(stones - move, False))
        return best
    else:
        best = float('inf')
        for move in range(1, 4):
            if stones - move >= 0:
                best = min(best, minimax(stones - move, True))
        return best

def best_move(stones):
    for move in range(1, 4):
        if stones - move >= 0 and minimax(stones - move, False) == 1:
            return move
    return 1

stones = 10
print(f"Starting Nim game with {stones} stones")

while stones > 0:

    move = int(input(f"Your turn (1-3 stones, remaining {stones}): "))
    stones -= move
    if stones == 0:
        print("You win!")
        break

    ai = best_move(stones)
    stones -= ai
    print(f"AI removes {ai} stones, remaining {stones}")
    if stones == 0:
        print("AI wins!")
        break
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\dax
p1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 59084 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p2
8.py
Starting Nim game with 10 stones
Your turn (1-3 stones, remaining 10): 3
AI removes 3 stones, remaining 4
Your turn (1-3 stones, remaining 4): 2
AI removes 2 stones, remaining 0
AI wins!
```

29. Develop an A search algorithm for finding the shortest route between locations with given heuristics.

City	Heuristic (h) to Goal
A	10
B	6
C	4
D	7
E	2
Goal	0

```
import heapq

graph = {
   "A": {"B": 1, "C": 3},
   "B": {"D": 3, "E": 7},
   "C": {"D": 1, "E": 4},
   "D": {"Goal": 2},
   "E": {"Goal": 1},
   "Goal": {}
}

heuristic = {"A": 10, "B": 6, "C": 4, "D": 7, "E": 2, "Goal": 0}

def a_star(start, goal):
   open_set = []
   heapq.heappush(open_set, (heuristic[start], 0, start, [start]))  # (f, g, node, path)

   while open_set:
      f, g, node, path = heapq.heappop(open_set)
      if node == goal:
         return path, g
      for neighbor, cost in graph[node].items():
         g_new = g + cost
         f_new = g_new + heuristic[neighbor]
         heapq.heappush(open_set, (f_new, g_new, neighbor, path + [neighbor]))
   return None

path, total_cost = a_star("A", "Goal")
print("Shortest path:", path)
print("Total cost:", total_cost)
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 59104 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p29.py
Shortest path: ['A', 'C', 'E', 'Goal']
Total cost: 8
```

29. Implement Alpha-Beta Pruning in a simple game tree (like a reduced Tic-Tac-Toe tree).

```
def alpha_beta(node, depth, alpha, beta, maximizingPlayer):
   if depth == 0 or isinstance(node, int):  # Leaf node
      return node

   if maximizingPlayer:
      maxEval = float('-inf')
```

```
        for child in node:
            eval = alpha_beta(child, depth-1, alpha, beta, False)
            maxEval = max(maxEval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                break
        return maxEval
    else:
        minEval = float('inf')
        for child in node:
            eval = alpha_beta(child, depth-1, alpha, beta, True)
            minEval = min(minEval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                break
        return minEval

game_tree = [
    [[3, 5], [6, 9]],
]

result = alpha_beta(game_tree, 3, float('-inf'), float('inf'), True)
print("Best value for Max:", result)
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win
32-x64\\bundled\\libs\\debugpy\\launcher 59123 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p30.py
Best value for Max: 5

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

31. Simulate uncertain knowledge by assigning confidence levels to rules and use rule selection based on thresholds.

```
rules = [
    {"rule": "If it is raining, then take an umbrella", "confidence": 0.9},
    {"rule": "If it is sunny, then wear sunglasses", "confidence": 0.8},
    {"rule": "If it is cloudy, then take a jacket", "confidence": 0.6},
    {"rule": "If it is snowing, then wear boots", "confidence": 0.95},
]

threshold = 0.7
selected_rules = [r["rule"] for r in rules if r["confidence"] >= threshold]

print("Rules selected based on threshold:", threshold)
for r in selected_rules:
    print("-", r)
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win
32-x64\\bundled\\libs\\debugpy\\launcher 59148 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p31.py
Rules selected based on threshold: 0.7
- If it is raining, then take an umbrella
- If it is sunny, then wear sunglasses
- If it is snowing, then wear boots

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

32. Model the Tweety example using exceptions and rules to handle uncertainty (e.g., birds generally fly unless they are penguins).

```
animals = {
    "Tweety": "bird",
    "Pingu": "penguin",
    "Eagle": "bird"
}

def can_fly(animal):
    try:
        if animals[animal] == "penguin":
            raise Exception(f"{animal} is a penguin and cannot fly!")

        elif animals[animal] == "bird":
            return f"{animal} is a bird and can fly."
        else:
            return f"No rule for {animal}."
    except Exception as e:
        return str(e)

for animal in animals:
    print(can_fly(animal))
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\dax
p1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 59164 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p3
2.py
Tweety is a bird and can fly.
Pingu is a penguin and cannot fly!
Eagle is a bird and can fly.

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
```

33. Implement a rule-based system where multiple causes can lead to a symptom, and the program ranks possible causes.

```
rules = [
    {"cause": "Flu", "symptom": "Fever", "confidence": 0.9},
    {"cause": "Cold", "symptom": "Fever", "confidence": 0.6},
    {"cause": "Malaria", "symptom": "Fever", "confidence": 0.95},
    {"cause": "Flu", "symptom": "Cough", "confidence": 0.8},
    {"cause": "Cold", "symptom": "Cough", "confidence": 0.7},
]

observed_symptom = "Fever"

possible_causes = [r for r in rules if r["symptom"] == observed_symptom]
```

```
ranked_causes = sorted(possible_causes, key=lambda x: x["confidence"],
reverse=True)

print(f"Possible causes for {observed_symptom}:")
for r in ranked_causes:
    print(f"{r['cause']} (confidence: {r['confidence']})")
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\dax
p1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 59192 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p3
3.py
Possible causes for Fever:
Malaria (confidence: 0.95)
Flu (confidence: 0.9)
Cold (confidence: 0.6)
```

34. Design a simplified Bayesian-like reasoning system using rule weights or scoring system (simulated in PROLOG).

```
rules = [
    {"hypothesis": "Flu", "symptom": "Fever", "weight": 0.8},
    {"hypothesis": "Flu", "symptom": "Cough", "weight": 0.7},
    {"hypothesis": "Cold", "symptom": "Fever", "weight": 0.5},
    {"hypothesis": "Cold", "symptom": "Cough", "weight": 0.9},
    {"hypothesis": "Malaria", "symptom": "Fever", "weight": 0.95},
]

observed_symptoms = ["Fever", "Cough"]

scores = {}
for rule in rules:
    if rule["symptom"] in observed_symptoms:
        scores[rule["hypothesis"]] = scores.get(rule["hypothesis"], 0) + rule["weight"]

ranked_hypotheses = sorted(scores.items(), key=lambda x: x[1], reverse=True)

print("Hypotheses ranked by likelihood:")
for hyp, score in ranked_hypotheses:
    print(f"{hyp} (score: {score})")
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\daxp1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win
32-x64\\bundled\\libs\\debugpy\\launcher 59211 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p34.py
Hypotheses ranked by likelihood:
Flu (score: 1.5)
Cold (score: 1.4)
Malaria (score: 0.95)

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```

35. Build a basic expert system for weather prediction using rule-based inference with uncertain facts (e.g., "if clouds and humidity are high, rain is likely").

```
facts = {
    "clouds": 0.9,
    "humidity": 0.8,
    "windy": 0.4,
    "temperature_low": 0.3
}

rules = [
    {"prediction": "Rain", "conditions": ["clouds", "humidity"], "weight": 0.8},
    {"prediction": "Storm", "conditions": ["clouds", "windy"], "weight": 0.7},
    {"prediction": "Cold", "conditions": ["temperature_low"], "weight": 0.6},
    {"prediction": "Sunny", "conditions": [], "weight": 0.5} ]

predictions = []
for r in rules:
    if r["conditions"]:
        score = min(facts[c] for c in r["conditions"]) * r["weight"]
    else:
        score = r["weight"]
    predictions.append((r["prediction"], score))

predictions.sort(key=lambda x: x[1], reverse=True)

print("Weather predictions ranked:")
for pred, score in predictions:
    print(f"{pred} (score: {score:.2f})")
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI ; /usr/bin/env c:\\Users\\daxp1\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\dax
p1\\.vscode\\extensions\\ms-python.debugpy-2025.10.0-win32-x64\\bundled\\libs\\debugpy\\launcher 59228 -- C:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\AI\\p3
5.py
Weather predictions ranked:
Rain (score: 0.64)
Sunny (score: 0.50)
Storm (score: 0.28)
Cold (score: 0.18)

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/AI (master)
$
```